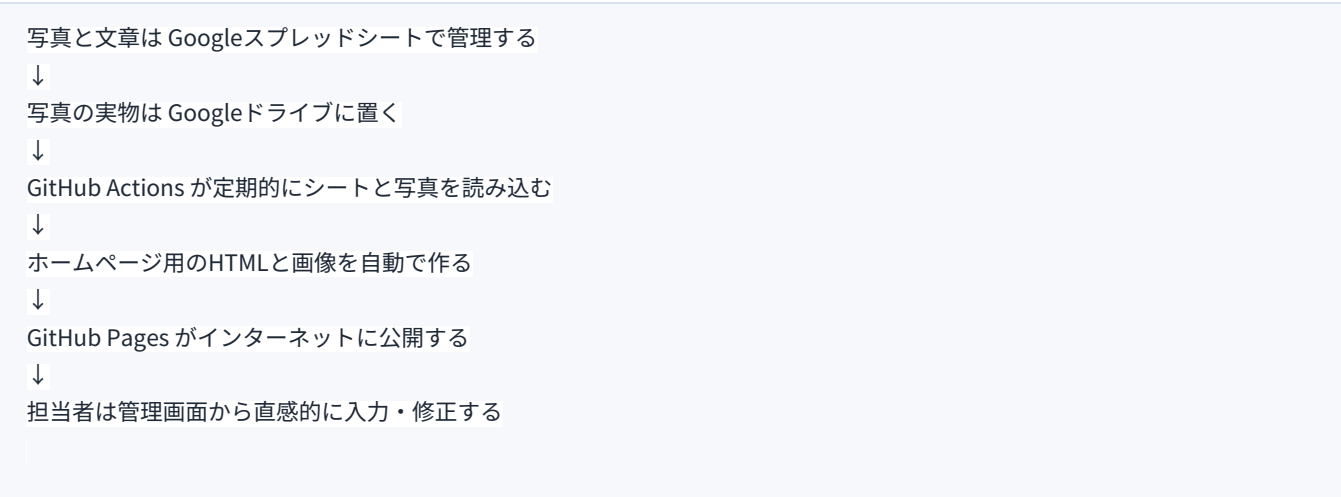


はじめての仕組み解説：ホームページと管理システム

SHIKA PHOTO LIBRARY を「何がどう動いているのか」から理解するための説明書です。
この資料は、GitHub、GitHub Actions、Googleスプレッドシート、Googleドライブ、Apps Script、管理画面、公開サイトの関係を、できるだけ専門用語をかみくだいて説明します。

0. まず結論

今回作ったものは、ひとことで言うと次の仕組みです。



つまり、担当者が普段触るのは **管理画面** です。ホームページのHTMLやプログラムを直接編集する必要はありません。

1. この仕組みで出てくる登場人物

名前	役割	たとえ
公開サイト	利用者が見るホームページ	店頭のショーケース
Googleスプレッドシート	写真名、説明、表示ON/OFFを管理する表	商品台帳
Googleドライブ	写真の実物を置く場所	写真倉庫
管理画面	担当者が入力しやすい操作画面	台帳入力用の専用フォーム
Apps Script	管理画面を動かすGoogle側の仕組み	スプレッドシート専用の小さなプログラム
GitHub	ホームページのファイル置き場	制作物の保管庫

GitHub Desktop	GitHubへ変更を送るためのパソコンアプリ	宅配便の受付窓口
GitHub Actions	自動でサイトを作り直す処理	自動作業ロボット
GitHub Pages	GitHub上のファイルをWeb公開する機能	公開用サーバー

2. 専門用語を先にざっくり説明

HTML

ホームページの文章や構造を書いたファイルです。

例えるなら、紙面の「見出し」「本文」「写真を置く場所」を決める設計図です。

CSS

ホームページの見た目を決めるファイルです。

文字の大きさ、色、余白、ボタンの形などを決めます。

JavaScript

ホームページ上で動きをつけるプログラムです。

検索、絞り込み、写真のクリック、ダウンロード確認画面などに使っています。

JSON

データを機械が読みやすい形で保存するファイルです。

今回なら「写真ID」「タイトル」「スポット名」「画像パス」などが入っています。

リポジトリ

GitHub上のプロジェクト置き場です。

このホームページでは photo-library というリポジトリに、サイトのファイルや自動更新の設定が入っています。

コミット

変更をひとまとまりとして記録することです。

「この時点ではこう直した」という履歴になります。

プッシュ

自分のパソコンにある変更をGitHubへ送ることです。

GitHub Desktopの `Push origin` がこれです。

ワークフロー

GitHub Actionsで実行する自動作業の流れです。
今回なら「スプレッドシートを読む → 写真を処理する → HTMLを作る → 公開する」という流れです。

デプロイ

作ったものを使える状態に公開することです。
Apps Scriptの管理画面をURLで開けるようにする操作も「デプロイ」と呼びます。

3. 全体の流れをもう少し詳しく

通常運用の流れ

1. 担当者が管理画面を開く
 2. 写真を登録する、または既存写真を修正する
 3. 管理画面がスプレッドシートへ保存する
 4. GitHub Actions が決まった時刻に自動実行される
 5. シートの内容を読み込む
 6. Googleドライブから写真を取得する
 7. 必要なサイズの画像を作る
 8. HTMLページを作る
 9. GitHub Pagesへ公開される
 10. 利用者がサイトで見る

すぐ公開したい場合

通常は自動更新を待ちます。急ぎの場合は、管理者がGitHubのActions画面で Run workflow を押します。

4. なぜスプレッドシートを使っているのか

ホームページを直接編集する方式だと、担当者がHTMLやGitHubを理解する必要があります。
一方、スプレッドシートを使うと、担当者は表や管理画面を触るだけで済みます。

方式	メリット	デメリット
HTMLを直接編集	自由度が高い	壊しやすい。専門知識が必要
スプレッドシート管理	誰でも更新しやすい	自動変換の仕組みが必要
管理画面管理	さらに直感的	最初にApps Scriptの設置が必要

今回の仕組みは、スプレッドシート管理に管理画面をかぶせています。

担当者に見せる部分は簡単にし、裏側でスプレッドシートとGitHubが動くようにしています。

5. Googleスプレッドシートの役割

スプレッドシートは、サイトの内容を管理する台帳です。
主なシートは次のとおりです。

シート	内容
写真	写真1枚ごとの情報
スポット	観光スポットページの説明文
サイト設定	トップページの文言や共通設定
シーン	トップページのおすすめシーン
季節	四季の写真設定
フォーム回答	投稿フォームなどから入った未確認データ
選択肢	プルダウンの候補

写真シートで大事な列

列	意味
id	写真を識別する番号。例：P0001
title	写真のキャプション
spot	どのスポットの写真か
tags	検索用のキーワード
driveFileId	Googleドライブ上の写真ID
publish	公開サイトに出すか
downloadAllowed	ダウンロード対象にするか

publish と downloadAllowed が両方TRUEの写真だけ、公開サイトに表示される運用です。

6. Googleドライブの役割

Googleドライブは、写真の実物を置く場所です。

スプレッドシートには写真そのものを入れるのではなく、写真のIDや共有リンクを入れます。

理由は、写真はファイルサイズが大きいからです。スプレッドシートには情報だけを入れ、実物はドライブに置く方が安定します。

Drive File IDとは

Googleドライブの写真リンクには、写真ごとのIDが入っています。

```
https://drive.google.com/file/d/ここがID/view
```

このIDを使って、GitHub Actionsが写真を取りに行きます。

7. 管理画面の役割

管理画面は、スプレッドシートを直接触らなくてもよいように作った入力画面です。

担当者は次の操作ができます。

タブ	できること
写真登録	新しい写真を登録（大きな画像は自動で縮小してから送られる）
写真管理	写真を検索、プレビュー、表示ON/OFF、キャプション修正
承認	未公開写真を公開待ちにする。登録日の新しい順・和暦表示で並び、日付で絞り込んだり、複数選択してまとめて公開／非公開にしたりできる
スポット	観光スポットの説明文を修正

管理画面はどこで動いているか

管理画面はGoogle Apps Scriptで動いています。

Apps Scriptは、GoogleスプレッドシートやGoogleドライブを操作できるGoogle公式のプログラム環境です。

今回の管理画面は、次の2ファイルでできています。

ファイル	役割
管理画面.gs	スプレッドシートへ読み書きする処理
管理画面.html	画面の見た目とボタン操作

サイドバー版とWebアプリ版

管理画面には2つの開き方があります。

開き方	説明
スプレッドシートの右側サイドバー	シート上部メニューから開く
独立したWebアプリURL	URLを開くだけで管理画面だけ表示

担当者には、WebアプリURLを共有するのが一番わかりやすいです。

8. GitHubの役割

GitHubは、ホームページのファイルを置く場所です。

今回のサイトでは、次のようなファイルがGitHubに入っています。

場所	内容
site/	公開サイト本体
site/scripts/	サイトを作るプログラム
site/data/	スプレッドシートから作られたデータ
site/assets/	画像、CSS、JavaScript
.github/workflows/	自動更新の設定
_source/photos/	元画像
docs/	マニュアル類

GitHub Desktopとは

GitHub Desktopは、GitHubへ変更を送るためのアプリです。

パソコン内でファイルを直しただけでは、GitHub上のファイルは変わりません。

パソコンで修正
↓ Commit
変更を履歴として記録
↓ Push
GitHubへ送る

この Commit と Push が終わって初めて、GitHub側に変更が届きます。

9. GitHub Actionsの役割

GitHub Actionsは、GitHub上で自動作業をする仕組みです。

今回のActionsは、次の作業をします。

1. リポジトリを取得
2. Pythonを用意
3. スプレッドシートを読み込む
4. Googleドライブから写真を取得
5. 写真をリサイズ
6. JSONデータを作る
7. HTMLページを作る
8. GitHub Pagesへ公開

Pythonとは

Pythonはプログラミング言語です。

今回のサイトでは、スプレッドシートの内容を読み込んだり、写真をリサイズしたり、HTMLを作ったりする作業に使っています。

Actionsが赤くなるとは

Actionsの実行結果が赤いバツになることがあります。

これは「自動作業の途中でエラーが起きた」という意味です。

よくある原因は次のとおりです。

原因	内容
シートが読めない	共有設定やSHEET_IDの問題
列名が変わった	プログラムが必要な列を見つけられない

写真が読めない	Drive共有、壊れた画像、ID間違い
データが少なすぎる	安全装置が止めている

10. GitHub Pagesの役割

GitHub Pagesは、GitHub上のHTMLファイルをインターネットに公開する機能です。
今回の公開サイトURLは次です。

```
https://shika-town.github.io/photo-library/
```

GitHub Actionsが作ったファイルを、GitHub Pagesが公開しています。

11. 1から作る場合の大きな流れ

ここからは、同じ仕組みを1から作るならどう進めるかを説明します。

Step 1. 何を管理したいか決める

最初に、サイトで管理する情報を決めます。
今回なら次です。

情報	例
写真	画像ファイル、タイトル、タグ、撮影場所
観光スポット	名称、説明文、住所、基本情報
トップページ	キャッチコピー、大きな写真、人気タグ
公開制御	表示ON/OFF、ダウンロード対象ON/OFF

この段階では、デザインよりも「何を後から更新したいか」を決めるのが大切です。

Step 2. スプレッドシートの形を決める

次に、スプレッドシートの列を決めます。
例えば写真なら、次のような列が必要です。

```
id, title, spot, area, season, tags, sortOrder, driveFileId, description, publish, downloadAllowed
```


列名はプログラムが読むため、むやみに変えない方が安全です。

Step 3. ホームページ本体を作る

HTML、CSS、JavaScriptでホームページを作ります。

この時点では、仮の写真や仮の文章で作っても大丈夫です。

重要なのは、あとからデータを差し替えられる構造にすることです。

Step 4. データ生成スクリプトを作る

スプレッドシートからデータを読み、サイト用のJSONやHTMLを作るプログラムを作ります。

今回の主なスクリプトは次のような役割です。

スクリプト	役割
import-sheet.py	スプレッドシートを読み、写真をリサイズし、dataを作る
build-all.py	サイト全体を作り直す
build-photos.py	写真詳細ページを作る
build-spots.py	スポットページを作る
build-library.py	写真一覧データを作る
build-seo.py	sitemapなど検索エンジン向けファイルを作る

Step 5. GitHubへ置く

ローカルで作ったサイトをGitHubリポジトリへ置きます。

この時点で、GitHub Desktopを使って **Commit** と **Push** をします。

Step 6. GitHub Pagesを有効にする

GitHubのSettingsからPagesを有効にします。

今回のようにActionsで公開する場合は、PagesのSourceをGitHub Actionsにします。

Step 7. GitHub Actionsを設定する

`.github/workflows/publish.yml` のような設定ファイルを作ります。

このファイルに「いつ、何を、自動で実行するか」を書きます。

今回なら、平日9時・12時・17時の自動更新と、手動実行ができる設定になっています。

Step 8. SHEET_IDを登録する

GitHub Actionsがどのスプレッドシートを読むかを知るため、GitHubに SHEET_ID を登録します。
SHEET_IDとは、スプレッドシートURLの /d/ と /edit の間にある文字列です。

```
https://docs.google.com/spreadsheets/d/ここがSHEET_ID/edit
```

Step 9. Apps Scriptで管理画面を作る

スプレッドシートに紐づくApps Scriptに、管理画面のファイルを入れます。

```
管理画面.gs  
管理画面.html
```

管理画面.gs はスプレッドシートに読み書きします。

管理画面.html は担当者が見る画面です。

Step 10. Webアプリとしてデプロイする

Apps ScriptをWebアプリとしてデプロイすると、管理画面だけをURLで開けます。

担当者には、このURLを共有します。

Step 11. 運用テストをする

最低限、次を確認します。

確認	内容
写真登録	新しい写真が非公開で入るか
写真管理	プレビュー、ON/OFF、キャプション修正ができるか
承認	未公開写真を公開待ちにできるか
自動更新	GitHub Actionsが緑になるか
公開サイト	写真が想定どおり表示されるか

12. 今回作った管理画面で実際に起きていること

写真管理で写真を選ぶ

管理画面がスプレッドシートの写真一覧を読み込みます。
写真ID、キャプション、スポット名、プレビューURLを画面に渡します。

表示ON/OFFを変える

「公開サイトに表示する」をOFFにすると、スプレッドシートの publish がFALSEになります。
次回のサイト生成時、その写真は除外されます。

ダウンロード対象を変える

「ダウンロード対象にする」をOFFにすると、downloadAllowed がFALSEになります。
今回の運用では、ダウンロード対象ではない写真は公開サイトに出さない方針です。

キャプションを変える

キャプションを保存すると、スプレッドシートの title が変わります。
次回のサイト生成時、写真カードやスポットページに反映されます。

13. 公開中サイトを壊さないための安全装置

今回の仕組みには、いくつか安全側の考え方が入っています。

安全策	内容
新規写真は非公開	登録しただけでは公開されない
publishで非表示	削除せずに戻せる
downloadAllowedで制御	ダウンロード対象だけ出せる
Actions失敗時は公開しない	エラー時に壊れたサイトを出しにくい
画像が読めない時の回避	既存画像を再利用、または該当写真を飛ばす
件数が少なすぎる時は停止	シート読み込み失敗で空サイトになるのを防ぐ

14. 普段の運用で覚えること

担当者は、次だけ覚えれば大丈夫です。

管理画面を開く
写真を選ぶ
プレビューを見る

必要な項目を直す
保存する
反映を待つ

管理者は、次を覚えておくで安心です。

Apps Scriptを更新する方法
Webアプリを新バージョンでデプロイする方法
GitHub Actionsを手動実行する方法
赤いエラーのログを見る方法
緊急時にpublishをFALSEにする方法

15. よくある勘違い

保存したのにサイトが変わらない

管理画面で保存した時点では、スプレッドシートが変わっただけです。
公開サイトが変わるのは、GitHub Actionsが実行されて成功した後です。

GitHub DesktopでCommitしたのにサイトが変わらない

Commitはパソコン内の記録です。
GitHubへ送るにはPushが必要です。

Pushしたのにサイトが変わらない

Push後にGitHub Actionsが動き、成功する必要があります。
Actionsが赤い場合は公開されません。

Apps Scriptで保存したのにWebアプリが変わらない

Webアプリは、保存しただけでは既存URLに反映されないことがあります。
「デプロイを管理」から新バージョンでデプロイします。

16. トラブル時の見方

まず見る順番

1. 管理画面で保存できたか
2. スプレッドシートの値が変わっているか
3. GitHub Actionsが動いたか
4. Actionsが緑か赤か

5. 公開サイトの該当ページが変わったか

Actionsが赤い場合

赤い実行を開き、ログを見ます。

ログの最後の方に原因が出ていることが多いです。

ログの例	意味
シートを読み込めません	スプレッドシート共有やSHEET_IDの問題
列が見当たりません	シートの見出し行が変わった
cannot identify image file	画像ファイルが壊れている、またはDriveからHTMLが返った
公開対象の写真が少ない	シート読み込みに失敗している可能性

17. ファイル整理後の構成

現在のフォルダは、大きく次のように整理されています。

フォルダ	意味
site/	公開サイト本体。重要
site/scripts/	サイトを作るプログラム。重要
site/scripts/gas/	Apps Scriptに貼る管理画面ファイル
_source/photos/	元画像。重要
docs/	マニュアル・設計資料
output/pdf/	PDF出力物
inbox/	追加したい写真の一時置き場
.github/workflows/	GitHub Actions設定。重要

18. 絶対に不用意に消さないもの

ファイル・フォルダ	理由
site/	公開サイト本体
_source/photos/	元画像
.github/workflows/publish.yml	自動更新の設定
site/scripts/import-sheet.py	シート読み込みの中心
site/scripts/build-all.py	サイト生成の中心
site/scripts/gas/管理画面.gs	管理画面の裏側
site/scripts/gas/管理画面.html	管理画面の見た目

19. 消してよいもの、残すもの

消してよい一時ファイル

種類	説明
.DS_Store	Macが自動で作る表示設定ファイル
pycache	Pythonが自動で作る一時フォルダ
*.pyc	Pythonの一時ファイル
.write-test	動作確認用の一時ファイル

基本的に残す資料

docs/ に入っている資料は、すぐ使わないものでも履歴として残す方が安全です。

特に、担当者マニュアルと管理者マニュアルは今後の引き継ぎに使います。

20. 今後の改善案

さらに使いやすくするなら、次の改善が考えられます。

改善	内容
権限別画面	担当者は写真管理だけ、管理者は全機能を表示
変更履歴表示	誰がいつ何を変えたか管理画面に出す
公開前チェック	人物・権利確認のチェック項目を追加
一括操作の拡張	承認タブの複数選択・まとめて公開／非公開は実装済み。写真管理タブでのまとめてタグ付けなどはまだ未対応
管理画面URLの固定案内	担当者用ブックマークとして周知

21. まとめ

今回の仕組みは、見た目としてはホームページですが、裏側では次の4つが連携しています。

Googleスプレッドシート
Googleドライブ
GitHub Actions
GitHub Pages

担当者は管理画面を使います。

管理者はApps ScriptとGitHub Actionsを保守します。

公開サイトは、GitHub Actionsが成功した時だけ更新されます。

これを理解しておけば、何か問題が起きても「どこで止まっているか」を順番に見つけられます。

最初は難しく感じますが、日常作業として覚えることは多くありません。

担当者：管理画面で保存する
管理者：必要なときにActionsとApps Scriptを見る

この2つに分けて考えると、かなり扱いやすくなります。